SIMATS ENGINEERING

ASSESSMENT TOOL 1: Experiments

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications.* (BL3)

Assessment Tool Weightage: 40%

Dave's Taxonomy: Precision (Level 3) **Total Marks: 40**
Question Count: 5

Code of Conduct

I **JAIVANT.S (192311326)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ***JAIVANT.S***

Assessment Tasks

Experiments

1. Experiment 1: Create and document a cloud storage comparison between object, block, and file storage. Select the best storage model for backups, VM disks, and shared enterprise documents.
2. Experiment 2: Develop a simple REST API workflow for a cloud-based student portal. Show request, response, authentication, and deployment flow.
3. Experiment 3: Design a serverless event flow for image upload processing using object storage, function-as-a-service, and notification service.
4. Experiment 4: Compare edge computing and fog computing for a smart traffic management system. Include architecture, latency considerations, and deployment location.
5. Experiment 5: Demonstrate a cloud application deployment workflow showing client access, browser interaction, web API call, storage access, and monitoring.

For each experiment, include objective, architecture sketch, procedure, expected output, and conclusion.

Experiments Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Experimental Design	25%	Design is systematic and technically strong	Mostly correct design	Basic design with gaps	Poorly designed activity
Use of Tools / Platforms	20%	Appropriate and effective use of cloud tools	Good tool usage with minor issues	Limited tool usage	Incorrect or missing tool usage
Application of	20%	Concepts are	Mostly accurate	Partial application	Weak application

Concepts		applied accurately to practical tasks	application		
Analysis and Output	20%	Strong analysis and meaningful outcome interpretation	Good analysis with minor gaps	Basic analysis	Little or no analysis
Documentation	15%	Clear, structured, and complete documentation	Mostly complete documentation	Partially complete	Poor documentation

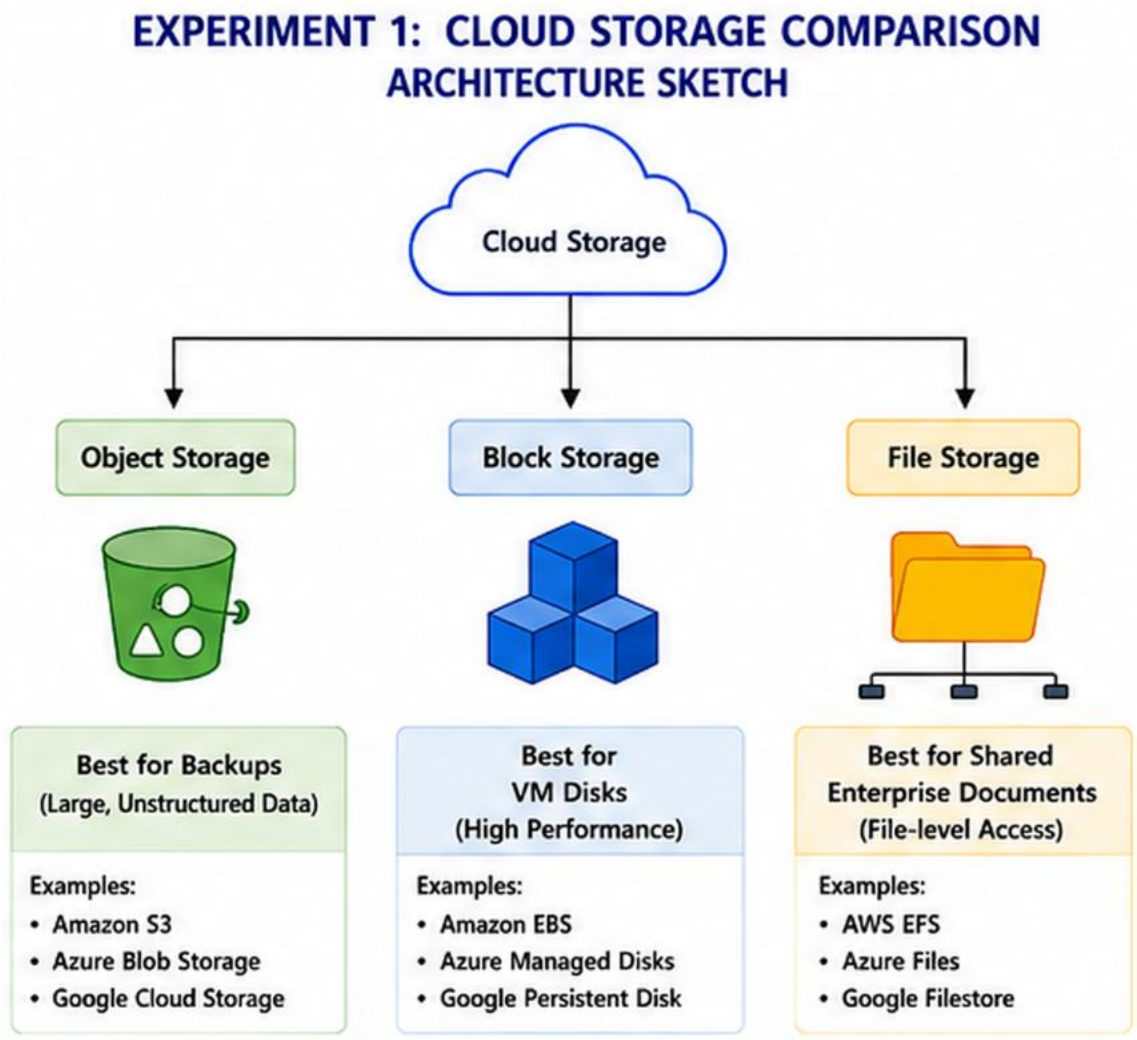
ANSWERS

EXPERIMENT 1: Cloud Storage Comparison

Objective

To compare **object storage, block storage, and file storage**, and identify the most suitable storage model for backups, virtual machine disks, and shared enterprise documents.

Architecture Sketch



Storage Comparison

Feature	Object Storage	Block Storage	File Storage
Data organization	Objects	Blocks	Files/Folders
Best suited for	Backups, media	VM disks	Shared documents
Access method	API/HTTP	Block-level	File system
Scalability	Very high	High	High
Sharing	Through applications	Usually attached to servers	Multiple users/servers
Example use	Backup files	Virtual machine disk	Enterprise file sharing

Procedure

1. Identify the storage requirements for backups, VM disks, and shared documents.
2. Compare object, block, and file storage based on performance, scalability, and accessibility.
3. Select the appropriate storage model for each requirement.
4. Map the selected storage model to a practical cloud deployment.
5. Document the comparison and justify the selections.

Expected Output

- **Backups → Object Storage:** Suitable because it provides scalable and durable storage for large amounts of backup data.
- **VM Disks → Block Storage:** Suitable because virtual machines require high-performance block-level storage.
- **Shared Enterprise Documents → File Storage:** Suitable because users and applications can access files through a shared file system.

Conclusion

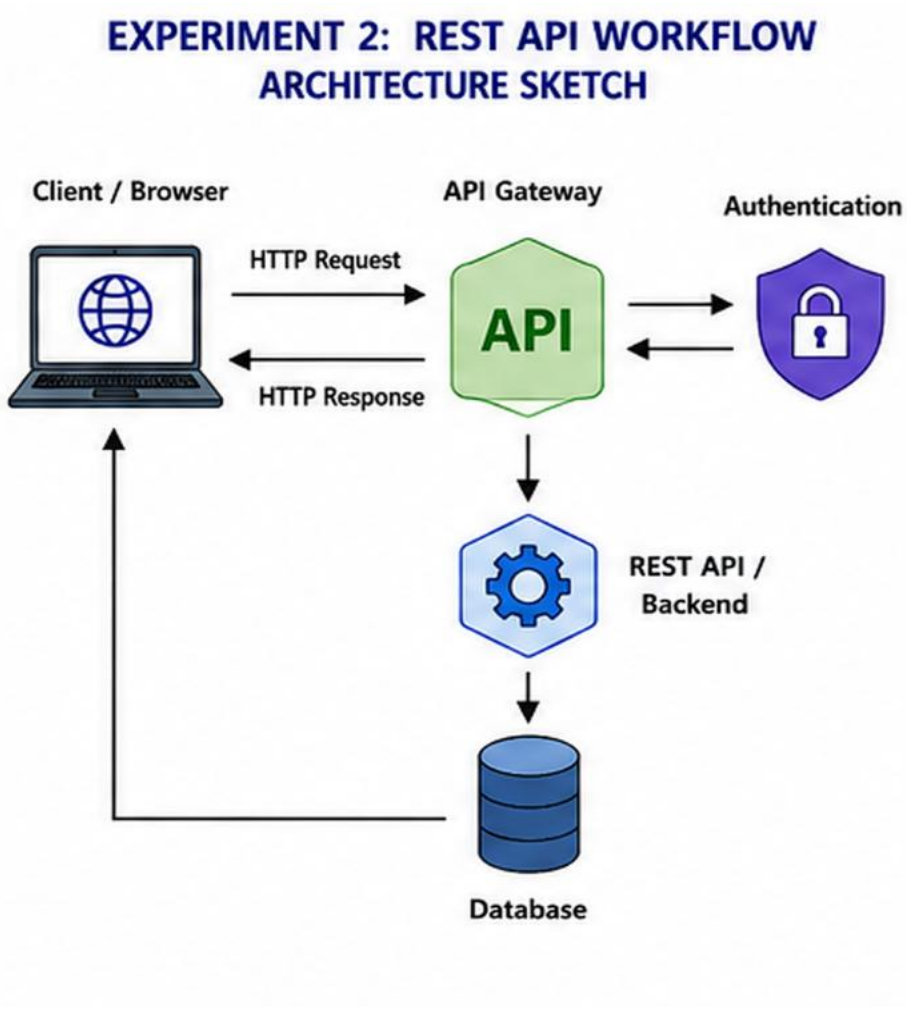
Object, block, and file storage are designed for different requirements. **Object storage is the best choice for backups, block storage for VM disks, and file storage for shared enterprise documents.** Selecting the correct storage model improves performance, scalability, and cost efficiency.

EXPERIMENT 2: REST API Workflow for a Cloud-Based Student Portal

Objective

To design a simple **REST API workflow** for a cloud-based student portal and demonstrate the request, response, authentication, and deployment flow.

Architecture Sketch



Procedure

1. Create a student portal backend that provides REST API endpoints.
2. Define endpoints such as:
    - GET /students
    - GET /students/{id}
    - POST /students
    - PUT /students/{id}

3. The student sends an HTTP request through the browser or application.
4. The API receives and validates the request.
5. Authentication is performed using credentials or an authentication token.
6. After successful authentication, the API accesses the required database information.
7. The server sends a response, normally in JSON format.
8. Deploy the API on a cloud platform so that it can be accessed through the Internet.

Example Request

GET /students/101

Authorization: Bearer <token>

Example Response

{

"student_id": 101,

"name": "Student",

"course": "Computer Science",

"status": "Active"

}

Expected Output

The student portal should successfully receive requests, authenticate users, retrieve the required student information, and return an appropriate response.

For example:

Browser → API Request → Authentication → Backend → Database

← JSON Response ←

Conclusion

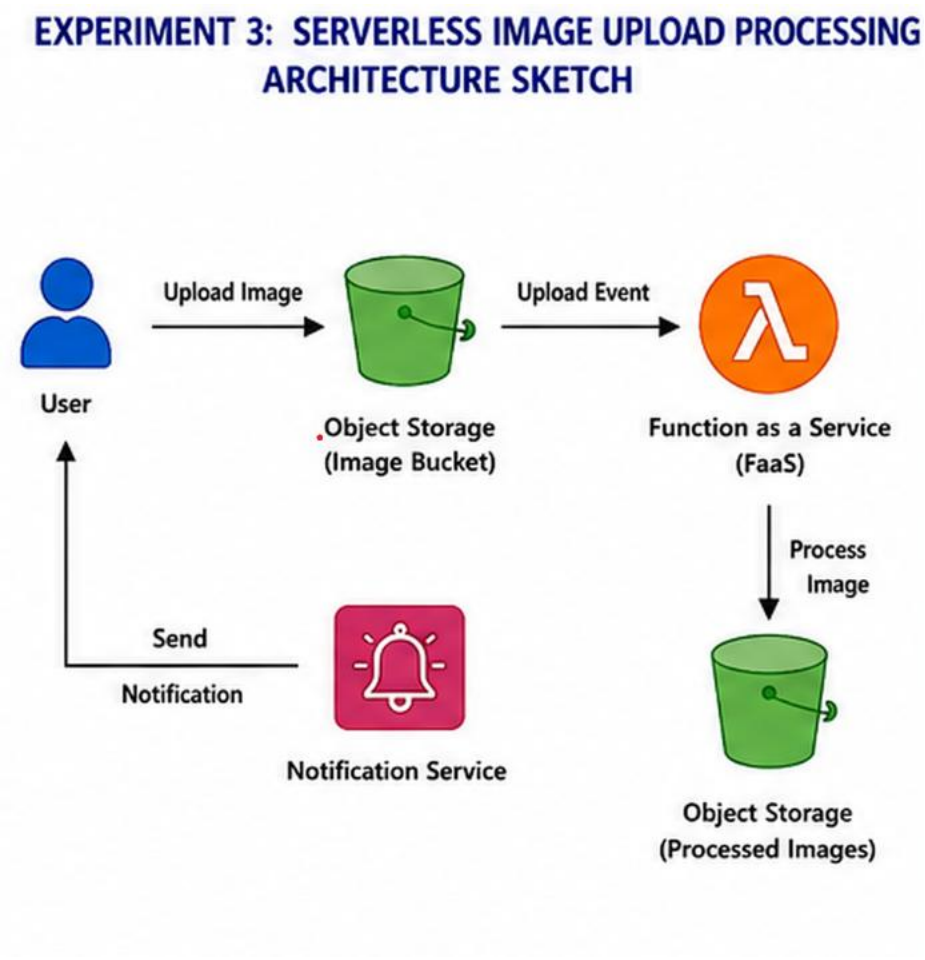
A REST API provides a simple and standard method for communication between the student portal and its backend. Authentication protects student information, while cloud deployment makes the API accessible and scalable.

EXPERIMENT 3: Serverless Image Upload Processing

Objective

To design a serverless event-driven workflow for processing uploaded images using **object storage, Function-as-a-Service (FaaS), and a notification service**.

Architecture Sketch



Procedure

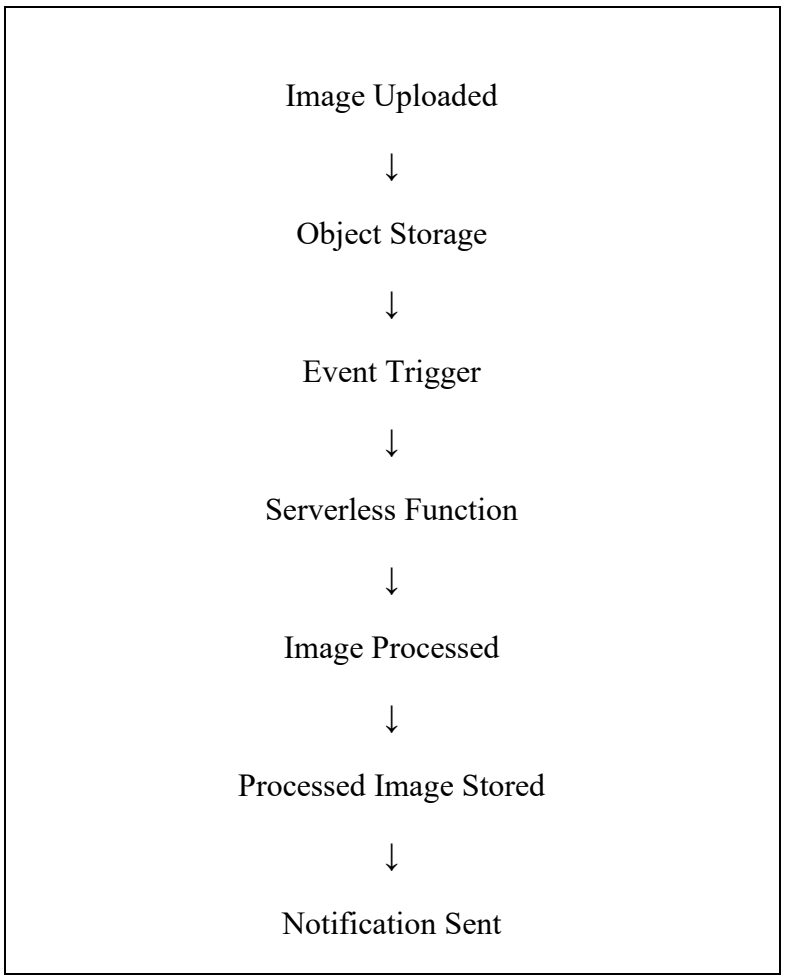
1. Create a cloud object storage bucket.
2. Allow users to upload images into the storage bucket.
3. Configure an event trigger for new image uploads.
4. Connect the event to a serverless function.
5. The function automatically starts when a new image is uploaded.
6. The function can perform operations such as:

- Image resizing.
- Thumbnail generation.
- Format conversion.
- Basic image validation.

7. Store the processed image back in object storage.
8. Use a notification service to inform the user or administrator that processing is complete.

Expected Output

When a user uploads an image:



The complete process should occur automatically without requiring a continuously running server.

Conclusion

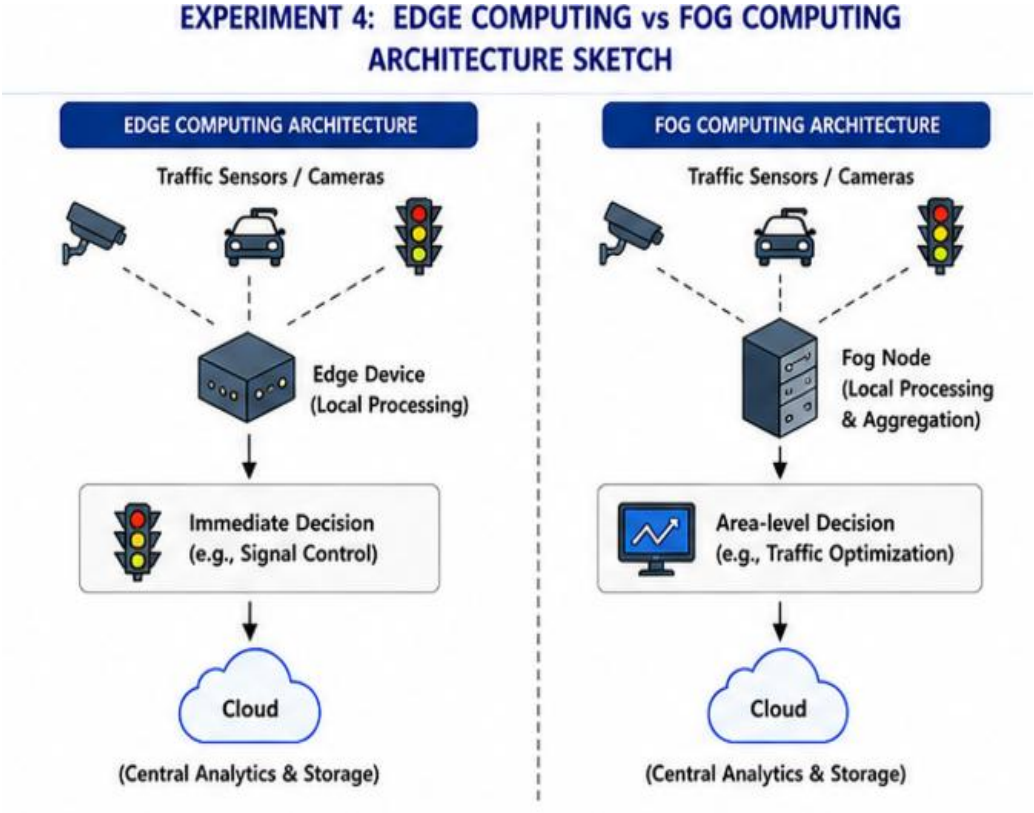
A serverless event-driven architecture is suitable for image processing because the function runs only when an upload event occurs. It reduces server management, automatically handles varying workloads, and provides an efficient way to process uploaded images.

EXPERIMENT 4: Edge Computing vs Fog Computing for Smart Traffic

Objective

To compare **edge computing and fog computing** for a smart traffic management system based on architecture, latency, and deployment location.

Architecture Sketch



Comparison

Feature	Edge Computing	Fog Computing
Processing location	Directly near sensors/devices	Intermediate local network
Latency	Very low	Low
Main purpose	Immediate decisions	Local coordination and processing
Data aggregation	Limited	Can aggregate multiple devices
Example	Detecting an accident	Managing traffic across an area

Procedure

1. Install traffic sensors and cameras at roads and junctions.
2. Collect information such as vehicle count, speed, and traffic density.
3. In the **edge model**, process the information directly near the traffic sensor.
4. In the **fog model**, send information from multiple sensors to a nearby fog node.
5. Compare the time required to process the information.
6. Send selected information to the cloud for long-term storage and city-wide analytics.
7. Evaluate which model is more suitable for different traffic-management decisions.

Latency Considerations

Edge computing provides the **lowest latency** because processing occurs very close to the data source. This is useful for immediate decisions such as detecting accidents or controlling traffic signals.

Fog computing also provides low latency but adds an intermediate processing layer. It is useful when information from several nearby sensors needs to be combined before making a decision.

Deployment Location

- **Edge:** Traffic cameras, sensors, traffic lights, or nearby gateways.
- **Fog:** Local traffic control centers or network nodes serving a group of roads.
- **Cloud:** Central data centers for long-term storage, historical analysis, and city-wide prediction.

Expected Output

The system should be able to identify traffic conditions with low delay and select the appropriate processing location.

For example:

Sensor → Edge → Immediate Traffic Decision

Sensor Group → Fog → Area-Level Traffic Decision

All Data → Cloud → City-Wide Analytics

Conclusion

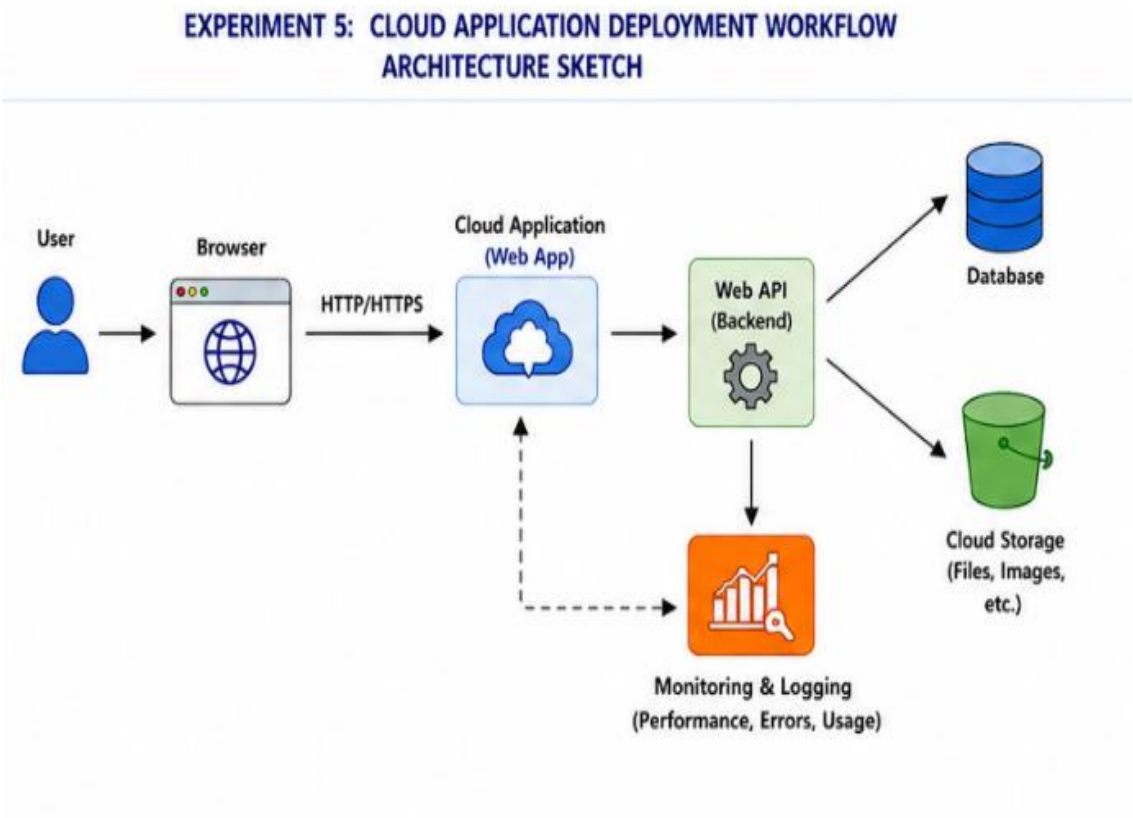
For a smart traffic system, both technologies can be useful. **Edge computing is better for extremely time-sensitive decisions**, while **fog computing is better for processing and coordinating information from multiple nearby devices**. A combination of Edge, Fog, and Cloud provides the most flexible solution.

EXPERIMENT 5: Cloud Application Deployment Workflow

Objective

To demonstrate a cloud application deployment workflow involving **client access, browser interaction, Web API communication, cloud storage, and monitoring**.

Architecture Sketch



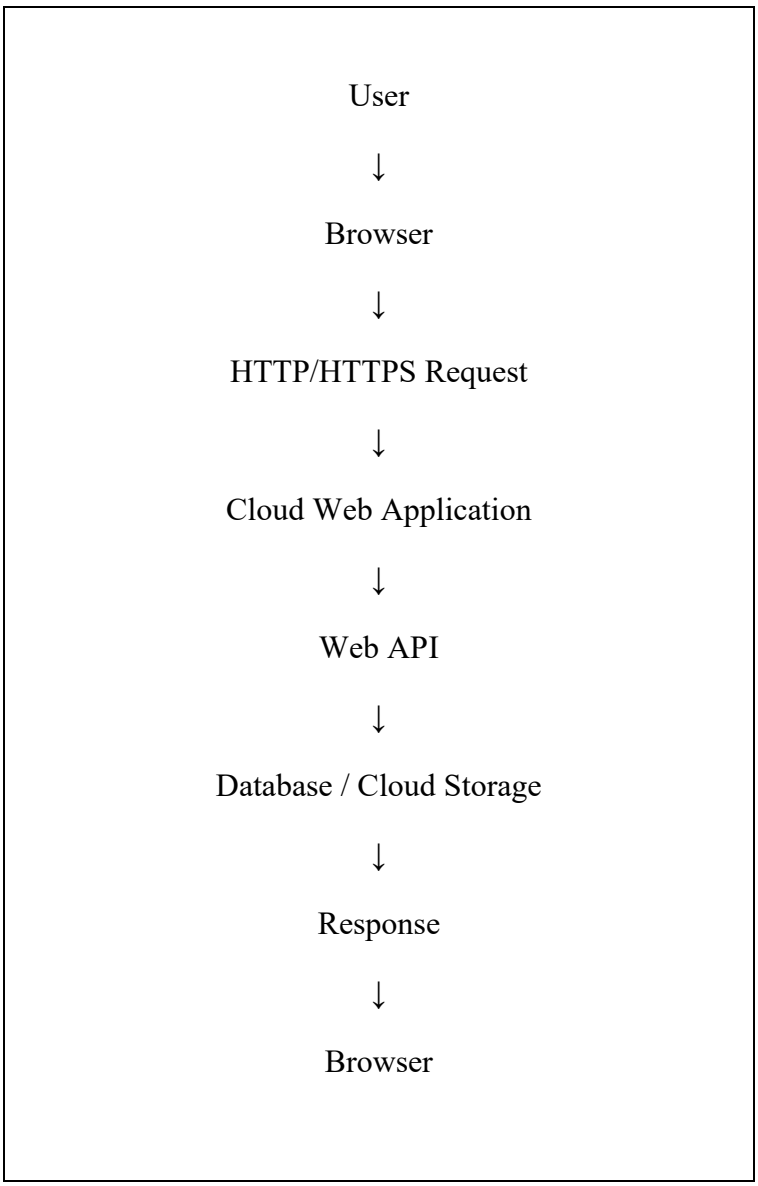
Procedure

1. Develop a simple cloud-based web application.
2. Deploy the application to a cloud environment.
3. The user opens the application using a web browser.
4. The browser sends an HTTP/HTTPS request to the cloud application.
5. The application communicates with backend services through a Web API.
6. The API retrieves or updates information in the cloud database.
7. Files such as images or documents are stored in cloud storage.
8. Monitoring tools are configured to track application performance, resource usage, and errors.

9. Test the application by sending requests and checking the returned responses.

Application Workflow

The complete workflow can be represented as:



At the same time, the monitoring system observes the application's performance and resource usage.

Expected Output

The deployed application should:

- Open successfully through a web browser.

- Accept user requests.
- Communicate with backend services through Web APIs.
- Read and write data to cloud storage or databases.
- Return the appropriate response to the user.
- Provide monitoring information about application performance.

Conclusion

A cloud application deployment combines browser-based client access, Web APIs, cloud storage, databases, and monitoring. This architecture provides **scalability, accessibility, reliability, and better visibility into application performance**, making it suitable for modern cloud-based applications.